

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчёт по курсовой работе

по предмету «Мобильная разработка (Android и IOS)»

Выполнил:

Игнатъев А.А.

К3440

Санкт-Петербург

2025 г.

Оглавление

1. Введение	3
2. Архитектура системы	5
1. Презентационный слой (Presentation Layer)	5
2. Слой бизнес-логики (Business/Domain Layer)	6
3. Слой данных (Data Layer)	6
4. Слой инфраструктуры (Infrastructure Layer)	7
3. Реализация функционала приложения	9
3.1 Архитектура модуля чатов	9
3.2 Система аутентификации	13
3.3 Криптографическая система	18
4. Выводы:.....	24

1. Введение

Данная курсовая работа посвящена разработке мобильного приложения под Android.

Цель – приобрести комплексный практический опыт в проектировании и разработке современных мобильных приложений для платформы Android. В ходе работы осваиваются и применяются ключевые инструменты и архитектурные шаблоны, включая компоненты Android Jetpack и паттерн Model-View-ViewModel (MVVM). Создаваемое приложение представляет собой упрощённый, но полнофункциональный клиент для мессенджера, реализующий следующий базовый пользовательский сценарий:

- **Управление профилем:** Возможность просмотра, редактирования и сохранения личной информации (имя, статус).
- **Кастомизация интерфейса:** Переключение между светлой и тёмной темами оформления приложения.
- **Работа с сообщениями:** Просмотр ленты диалогов и истории сообщений в хронологическом порядке.
- **Интерактивное общение:** Отправка и получение текстовых сообщений, а также мультимедийного контента (фото и видео), реализованное с использованием компонентов CameraX и ExoPlayer для захвата и воспроизведения медиа.
- **Энд-ту-энд шифрование:** Организация защищённого обмена сообщениями путём комбинирования симметричного (AES-256) и асимметричного (RSA) шифрования. При этом для каждого сообщения генерируется уникальный сеансовый AES-ключ, который передаётся получателю, зашифрованный его открытым RSA-ключом.

Технологический стек и архитектура базируются на среде разработки Android Studio и языке Kotlin. Основу реализации составляют компоненты Android Jetpack:

- **ViewModel & LiveData:** Для хранения и реактивной передачи

UI-состояния, обеспечения независимости данных от жизненного цикла интерфейса и следования паттерну MVVM.

- **Navigation Component:** Для декларативного описания навигационного графа и удобного перехода между экранами (список чатов, конкретный диалог, профиль пользователя, экран съёмки медиа).
- **Room Persistence Library:** Для надёжного локального кэширования данных (сообщений, информации о пользователях, токенов авторизации) и обеспечения офлайн-доступа.
- **Paging 3:** Для эффективной поэтапной подгрузки и отображения длинных списков сообщений в диалоге, поддерживая как локальные (Room), так и удалённые (сеть) источники данных через механизм RemoteMediator.
- **Data Binding & View Binding:** Для типобезопасного доступа к элементам UI и упрощения кода фрагментов.
- **Dependency Injection (Hilt):** Для автоматического управления зависимостями (репозитории, источники данных, ViewModel), что повышает тестируемость и читаемость кода.

Сетевое взаимодействие организовано с использованием библиотеки **Retrofit** для структурированных HTTP-запросов к серверному API мессенджера. Все сетевые операции обрабатываются в фоновых потоках с последующей доставкой результатов в основной поток через Kotlin Coroutines Flow.

Особенности реализации безопасности:

- **Аутентификация:** Используется JWT-токен (Access/Refresh), хранящийся в защищённом SharedPreferences.
- **Шифрование сообщений:** Реализована гибридная модель. Каждое сообщение шифруется уникальным AES-256 ключом со случайным IV. Этот сеансовый ключ шифруется открытым RSA-ключом каждого участника диалога и отправляется на сервер вместе с зашифрованным

текстом. Расшифровка на стороне клиента происходит с использованием приватного RSA-ключа.

Таким образом, проект фокусируется на глубоком освоении **полного цикла разработки UI, управления данными** (от локальной БД до облачного API), **архитектурных паттернов (MVVM), реактивного программирования (Flow, LiveData)** и **криптографических основ** для создания безопасного мессенджера

2. Архитектура системы

1. Презентационный слой (Presentation Layer)

Структура и ответственность:

Презентационный слой организован по паттерну **MVVM** и отвечает за отображение пользовательского интерфейса и обработку пользовательских действий. Этот слой строго отделён от бизнес-логики и работает только с данными, предоставляемыми ViewModel.

Ключевые компоненты:

- **Fragments**

(View): ChatFragment, AuthFragment, UsersFragment, PhotoFragment, VideoFragment, RecorderFragment, ChatListFragment

- Отвечают за отображение UI
- Обработывают пользовательские события (клики, ввод текста)
- Наблюдают за изменениями в ViewModel через LiveData/Flow
- Управляют жизненным циклом UI компонентов

- **ViewModels:** ChatViewModel, AuthViewModel, UsersViewModel, ChatListViewModel

- Хранят и управляют UI-состоянием
- Предоставляют данные для View через LiveData/StateFlow
- Вызывают методы репозитория для выполнения операций
- Обработывают преобразование данных (Entity → Model)

- **Data Binding/View Binding:**

- Обеспечивают типобезопасный доступ к элементам UI
- Упрощают привязку данных между View и ViewModel

2. Слой бизнес-логики (Business/Domain Layer)

Структура и ответственность:

Этот слой содержит всю бизнес-логику приложения, инкапсулированную в репозиториях и use cases. Он полностью независим от Android framework и может быть протестирован без UI.

Ключевые компоненты:

- **Repositories:** ChatRepository, AuthRepository, ChatListRepository, Users Repository
 - Предоставляют единый интерфейс для доступа к данным
 - Определяют источники данных (локальный/удалённый)
 - Реализуют стратегии кэширования и синхронизации
 - Обработывают преобразование данных между слоями
- **Use Cases/Interactors:**
 - Инкапсулируют конкретные бизнес-правила
 - Координируют работу между репозиториями
 - Пример: шифрование/дешифрование сообщений в ChatRepository

3. Слой данных (Data Layer)

Структура и ответственность:

Слой данных отвечает за управление всеми источниками данных приложения, включая локальное хранилище, сетевые запросы и пользовательские настройки.

Подслой:

3.1 Локальное хранилище (Room Database):

- **DAOs (Data Access Objects):** ChatDao
 - Определяют SQL-запросы для доступа к данным
 - Поддерживают PagingSource для страничной загрузки
- **Entities:** MessageEntity

- Представляют таблицы в базе данных
- Аннотируются Room-аннотациями
- **Преимущества:** Офлайн-доступ, быстрый доступ к кэшированным данным

3.2 Сетевой слой (Retrofit):

- **API интерфейсы:** ServerApi
 - Определяют REST эндпоинты с помощью Retrofit-аннотаций
 - Возвращают suspend функции для корутин
- **DTO объекты (Data Transfer Objects):** MessageDto, UserDto, TokensDto
 - Сериализуются/десериализуются Gson
 - Соответствуют структуре API ответов

3.3 Локальные настройки:

- **SharedPreferences/DataStore:** AuthLocalDataSource
 - Хранение токенов авторизации
 - Хранение криптографических ключей
 - Настройки приложения

4. Слой инфраструктуры (Infrastructure Layer)

Специализированные модули:

4.1 Криптографический модуль:

- RSAUtils — генерация и управление RSA ключевой парой
- AesUtils — симметричное шифрование сообщений
- **Гибридная модель:** AES для сообщений + RSA для передачи ключей

4.2 Медиа-модуль:

- **CameraX** — захват фото и видео
- **ExoPlayer** — воспроизведение видео
- **Галерея** — предпросмотр медиаконтента

Поток данных в системе

Стандартный поток (отправка сообщения):

1. **Пользователь** вводит текст в ChatFragment

2. **ChatFragment** вызывает ChatViewModel.sendMessage()
3. **ChatViewModel** вызывает ChatRepository.sendMessage()
4. **ChatRepository** выполняет:
 - Генерация AES ключа и IV для сообщения
 - Шифрование текста AES-256
 - Шифрование AES ключа RSA открытыми ключами участников
 - Отправка через ChatCloudDataSource (Retrofit)
 - Сохранение в локальную БД через ChatDao
5. Данные возвращаются по цепочке, обновляя UI

Поток получения сообщений (Paging 3):

1. **ChatFragment** подписывается на ChatViewModel.messagesFlow
2. **ChatViewModel** инициализирует Pager с RemoteMediator
3. **RemoteMediator** управляет загрузкой:
 - Проверяет локальную БД (Room)
 - При необходимости загружает из сети
 - Объединяет результаты
4. Данные передаются через PagingData в адаптер

Принципы проектирования

1. Единая ответственность (Single Responsibility)

Каждый компонент выполняет одну чётко определённую задачу:

- View — отображение
- ViewModel — управление состоянием
- Repository — управление данными
- DAO — доступ к БД

2. Инверсия зависимостей (Dependency Inversion)

- Высокоуровневые модули не зависят от низкоуровневых
- Использование Hilt для внедрения зависимостей
- Интерфейсы определяют контракты между слоями

3. Реактивное программирование

- LiveData для UI состояния

- Flow для асинхронных операций
- Корутины для управления потоками

4. Безопасность как принцип

- Шифрование данных end-to-end
- Безопасное хранение ключей
- JWT-аутентификация с refresh токенами

На диаграмме ниже представлена общая схема архитектуры приложения, демонстрирующая основные компоненты и взаимодействие между ними:

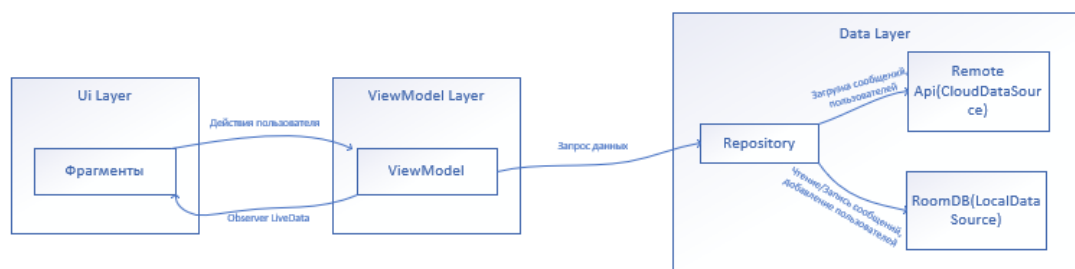


Рисунок 1 – общая структура системы

3. Реализация функционала приложения

3.1 Архитектура модуля чатов

3.1.1 Экран диалога (ChatFragment)

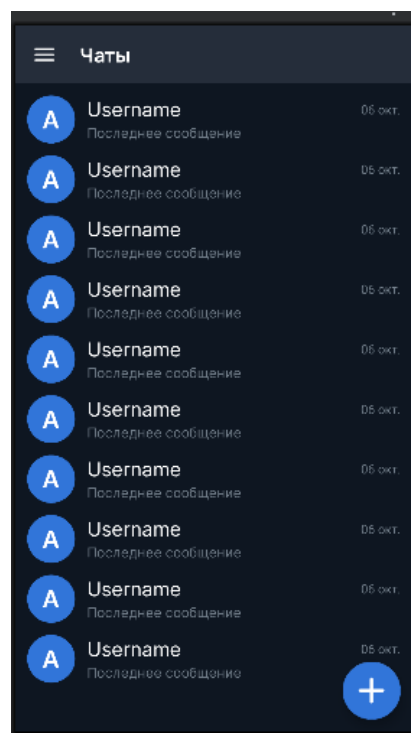


Рисунок 3.1: Интерфейс чата с историей сообщений

Архитектурный подход и структура управления: ChatFragment

представляет собой центральный элемент системы обмена сообщениями, построенный на принципах реактивного программирования и современных Android архитектурных паттернах. Фрагмент реализует сложную систему отображения истории переписки с поддержкой бесконечной прокрутки через Paging 3 и интегрированную панель ввода сообщений. Ключевой особенностью является разделение ответственности: UI-логика остается во фрагменте, а бизнес-логика делегируется ChatViewModel, что обеспечивает тестируемость и поддержку состояния при изменениях конфигурации.

Система обработки оконных инсетов: Реализация включает адаптивную настройку отступов для различных типов устройств и состояний системы. Панель инструментов получает отступ сверху, учитывающий статус бар, в то время как панель ввода динамически регулирует нижний отступ в зависимости от наличия клавиатуры. Этот подход гарантирует корректное отображение интерфейса на устройствах с вырезами экрана, жестовой навигацией и различными размерами дисплеев.

Механизм периодического обновления: Для обеспечения актуальности данных реализована фоновая синхронизация с интервалом 5 секунд. Система использует Kotlin корутины с привязкой к жизненному циклу view, что предотвращает утечки памяти и гарантирует остановку обновлений при переходе фрагмента в неактивное состояние. Этот подход позволяет пользователям видеть новые сообщения практически в реальном времени без необходимости ручного обновления.

@AndroidEntryPoint

```
class ChatFragment : Fragment() {  
    private lateinit var binding: FragmentChatBinding
```

```

private val args: ChatFragmentArgs by navArgs()
private val model: ChatViewModel by viewModels()
private val adapter = ChatAdapter()

override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    model.initPaging(args.peer)
    viewLifecycleOwner.lifecycleScope.launch {
        model.messagesFlow.collectLatest {
            adapter.submitData(it)
        }
    }

    binding.input.addTextChangedListener {
        binding.sendBtn.isVisible = it.toString().isNotEmpty()
    }

    binding.sendBtn.setOnClickListener {
        model.sendMessage(binding.input.text.toString(), args.peer)
        binding.input.setText("")
    }

    viewLifecycleOwner.lifecycleScope.launch {
        while (true) {
            delay(5000L)
            adapter.refresh()
        }
    }
}

```

Управление состоянием панели ввода: Реализована

интеллектуальная система отображения кнопки отправки, которая динамически появляется только при наличии текста в поле ввода. Это достигается через `TextWatcher`, отслеживающий изменения содержимого в реальном времени. При отправке сообщения поле немедленно очищается, обеспечивая мгновенную визуальную обратную связь и готовность к вводу следующего сообщения.

Архитектура управления потоком сообщений: ChatViewModel инкапсулирует сложную логику работы с историей переписки, интегрируя локальное хранилище Room, сетевой слой Retrofit и систему пагинации Paging 3. Использование паттерна Repository обеспечивает абстракцию над источниками данных, упрощая тестирование и поддержку кода. ViewModel управляет трансформацией данных из формата хранения (MessageEntity) в формат представления (MessageModel) с определением направления сообщений.

Конфигурация системы пагинации: Инициализация Pager включает настройку размера страницы (30 сообщений), что обеспечивает баланс между производительностью и отзывчивостью интерфейса. RemoteMediator используется для координации загрузки данных из сети и их сохранения в локальную базу данных. Этот подход позволяет реализовать бесконечную прокрутку с автоматической подгрузкой истории и эффективным кэшированием.

Определение направления сообщений: Ключевой особенностью является автоматическое определение принадлежности сообщений (входящие/исходящие) на основе сравнения ID отправителя с ID текущего пользователя. Эта логика выполняется на уровне ViewModel, что позволяет унифицировать обработку данных для UI и обеспечивает корректное отображение сообщений в соответствующих областях экрана.

```
@HiltViewModel
class ChatViewModel @Inject constructor(
    private val chatRepository: ChatRepository,
    private val authRepository: AuthRepository,
    private val chatDao: ChatDao
) : ViewModel() {
```

```

private lateinit var pager: Pager<Int, MessageEntity>
lateinit var messagesFlow: Flow<PagingData<MessageModel>>

@OptIn(ExperimentalPagingApi::class)
fun initPaging(peer: UserModel) {

    pager = Pager(
        config = PagingConfig(pageSize = 30),
        remoteMediator = ChatRemoteMediator(peer, chatRepository)
    ) {
        chatDao.getMessages(peer.userId)
    }

    val myId = authRepository.getAccessTokenModel().id
    messagesFlow = pager.flow.map { value: PagingData<MessageEntity> ->
        value.map { messageEntity ->
            messageEntity.toModel(myId)
        }
    }.cachedIn(viewModelScope)
}

fun sendMessage(text: String, peer: UserModel) {
    chatRepository.sendMessage(peer, text).launchIn(CoroutineScope(Dispatchers.IO))
}
}

```

Асинхронная обработка отправки сообщений: Метод отправки использует корутины с указанием `Dispatchers.IO` для выполнения сетевых операций в фоновом потоке. Это предотвращает блокировку основного потока UI и обеспечивает плавность интерфейса даже при медленных сетевых соединениях. Использование `CoroutineScope` гарантирует автоматическую отмену операций при уничтожении `ViewModel`.

3.2 Система аутентификации

3.2.1 Экран авторизации (AuthFragment)

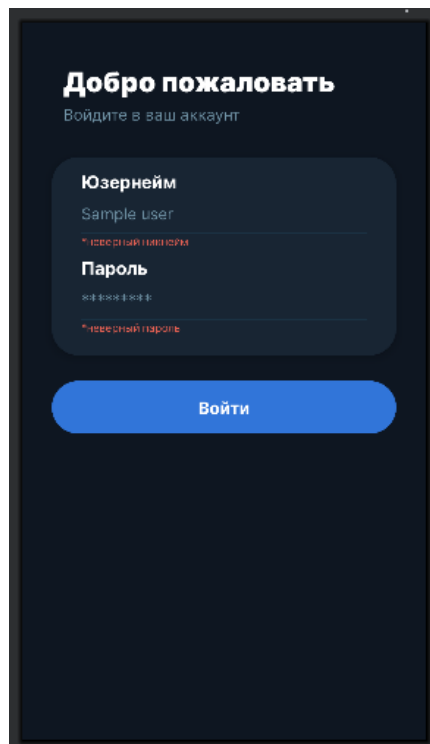


Рисунок 3.2: Интерфейс входа в систему

Архитектура управления пользовательским вводом и состоянием:

AuthFragment реализует комплексную систему обработки процесса аутентификации с акцентом на пользовательский опыт и безопасность. Фрагмент использует реактивную модель данных через StateFlow для управления UI состоянием, что позволяет динамически обновлять интерфейс в ответ на действия пользователя и результаты сетевых операций. При старте автоматически выполняется попытка обновления токенов, обеспечивая плавный переход для пользователей с активными сессиями.

Система валидации и обратной связи: Реализована многоуровневая валидация полей ввода с визуальными индикаторами ошибок. Система отслеживает состояние сетевого запроса и блокирует кнопку отправки во время выполнения операции, предотвращая множественные одновременные попытки аутентификации. Обработка ошибок включает преобразование технических сообщений об ошибках в понятные пользователю формулировки.

Адаптивная обработка системных окон: Фрагмент корректно работает на устройствах с различными типами экранов и версиями Android, динамически регулируя отступы с учетом статус бара, панели навигации и клавиатуры. Это обеспечивает оптимальное использование экранного пространства и предотвращает наложение элементов интерфейса.

```
@AndroidEntryPoint
class AuthFragment : Fragment() {
    private lateinit var binding: FragmentAuthBinding
    private val model: AuthViewModel by viewModels()

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        super.onCreateView(view, savedInstanceState)

        model.refreshTokens()

        viewLifecycleOwner.lifecycleScope.launch {
            model.uiState.collectLatest {
                updateUiState(it)
            }
        }

        binding.actionBtn.setOnClickListener {
            if (!model.uiState.value.isNetworkRequestActive) {
                model.signIn(
                    binding.usernameInput.text.toString(),
                    binding.passwordInput.text.toString()
                )
            }
        }
    }

    private fun updateUiState(state: AuthUiState) {
        if (state.errorMessage.isNotEmpty()) {
            showErrorAlert(state.errorMessage)
            model.clearMessages()
        }

        if (state.isLoggedIn) {
            findNavController().popBackStack()
        }
    }
}
```

```
        findNavController().navigate(NavGraphDirections.startChatListFragment())
    }

    binding.invalidUsernameText.isVisible = state.invalidUsername
    binding.invalidPasswordText.isVisible = state.invalidPassword
}
}
```

Автоматическая навигация при успешной аутентификации:

Система отслеживает состояние авторизации и автоматически перенаправляет пользователя на главный экран при успешном входе. Это устраняет необходимость ручных действий после завершения процесса аутентификации и улучшает общий пользовательский опыт. Навигация выполняется через Navigation Component с очисткой стека навигации для предотвращения возврата на экран входа.

3.2.2 ViewModel аутентификации

Управление состоянием процесса аутентификации:

AuthViewModel централизует всю бизнес-логику, связанную с входом в систему, используя MutableStateFlow для реализации одностороннего потока данных. Этот подход упрощает отладку и тестирование, обеспечивая предсказуемое поведение системы при различных сценариях. ViewModel интегрируется с репозиторием для выполнения операций аутентификации и управления токенами.

Многоуровневая система валидации: Перед инициированием сетевого запроса выполняется проверка заполненности обязательных полей с обновлением UI состояния для отображения соответствующих индикаторов ошибок. Валидация происходит как на стороне ViewModel (бизнес-логика), так и на стороне UI (визуальная обратная связь), обеспечивая комплексный подход к обеспечению качества данных.

Интеграция с системой обновления токенов: ViewModel

предоставляет механизм для автоматического обновления JWT токенов, что позволяет поддерживать длительные пользовательские

сессии без необходимости повторного ввода учетных данных. Эта функциональность критически важна для обеспечения удобства использования приложения и снижения частоты требований повторной аутентификации.

```
@HiltViewModel
class AuthViewModel @Inject constructor(
    private val authRepository: AuthRepository,
    @ApplicationContext private val context: Context
) : ViewModel() {
    private val _uiState = MutableStateFlow(
        AuthUiState(isLoggedIn = authRepository.isAuthenticated())
    )
    val uiState = _uiState.asStateFlow()

    fun signIn(username: String, password: String) {
        if (validateForm(username, password)) {
            _uiState.update { it.copy(isNetworkRequestActive = true) }
            authRepository.signIn(username, password).onEach { state ->
                if (state is UiState.Success) {
                    _uiState.update {
                        it.copy(isLoggedIn = true, isNetworkRequestActive = false)
                    }
                } else {
                    _uiState.update {
                        it.copy(
                            isLoggedIn = false,
                            errorMessage = (state as? UiState.Error)?.message ?: "",
                            isNetworkRequestActive = false
                        )
                    }
                }
            }.launchIn(viewModelScope)
        }
    }

    fun refreshTokens() {
        authRepository.refreshToken().launchIn(CoroutineScope(Dispatchers.IO))
    }
}
```

Обработка асинхронных операций и управление состоянием

загрузки: Все сетевые запросы выполняются в фоновых потоках с использованием корутин, что гарантирует отсутствие блокировок основного потока UI. ViewModel отслеживает состояние выполнения операций и соответствующим образом обновляет UI состояние, предоставляя пользователю визуальную обратную связь о ходе процесса аутентификации.

3.3 Криптографическая система

3.3.1 Гибридная система шифрования (AesUtils)

Архитектура симметричного шифрования: AesUtils реализует криптографическую защиту сообщений с использованием алгоритма AES-256 в режиме CBC с PKCS5Padding. Этот подход является отраслевым стандартом для обеспечения конфиденциальности и целостности данных. Система включает генерацию криптографически стойких случайных ключей и векторов инициализации для каждого сообщения, что реализует принцип совершенной прямой секретности.

Механизм управления сеансовыми ключами: Каждое сообщение шифруется уникальным сеансовым ключом, генерируемым случайным образом. Это означает, что компрометация одного ключа не ставит под угрозу безопасность других сообщений в истории переписки. Ключи генерируются с использованием криптографически безопасного генератора случайных чисел, обеспечивая высокий уровень энтропии.

Безопасная работа с бинарными данными и кодированием: Все криптографические операции используют Base64 кодирование для преобразования бинарных данных в текстовый формат, что обеспечивает корректную передачу через текстовые протоколы без потери или искажения информации. Система включает методы для конвертации между различными форматами представления ключей и векторов инициализации.

```
object AesUtils {
```

```

fun generateAESKey(): String {
    val keyGen = KeyGenerator.getInstance("AES")
    keyGen.init(256)
    val secretKey = keyGen.generateKey()
    return Base64.encodeToString(secretKey.encoded, Base64.NO_WRAP)
}

fun generateRandomIV(): ByteArray {
    return ByteArray(16) { Random.nextBytes(1)[0] }
}

fun encryptMessage(key: String, message: String, ivString: String): String {
    val secretKey = getSecretKeyFromString(key)
    val iv = ivFromString(ivString)
    val ivSpec = IvParameterSpec(iv)

    val cipher = Cipher.getInstance("AES/CBC/PKCS5Padding")
    cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivSpec)

    val encryptedBytes = cipher.doFinal(message.toByteArray(StandardCharsets.UTF_8))
    return Base64.encodeToString(encryptedBytes, Base64.NO_WRAP)
}

fun decryptMessage(key: String, encryptedMessage: String, ivString: String): String? {
    return try {
        val secretKey = getSecretKeyFromString(key)
        val iv = ivFromString(ivString)
        val ivSpec = IvParameterSpec(iv)

        val cipher = Cipher.getInstance("AES/CBC/PKCS5Padding")
        cipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec)

        val encryptedBytes = Base64.decode(encryptedMessage, Base64.DEFAULT)
        val decryptedBytes = cipher.doFinal(encryptedBytes)
        String(decryptedBytes, StandardCharsets.UTF_8)
    } catch (e: Exception) {
        null
    }
}

```

Обработка исключений и безопасное завершение операций: Все

методы дешифрования включают комплексную обработку исключений, что предотвращает краш приложения при получении поврежденных или неправильно зашифрованных данных. Вместо генерации исключений система возвращает null, позволяя вышележащим слоям предпринять соответствующие действия по обработке ошибки.

3.3.2 Система асимметричного шифрования (RSAUtils)

Архитектура управления ключевыми парами: RSAUtils реализует генерацию и использование RSA ключевых пар длиной 2048 бит, что соответствует современным стандартам безопасности для асимметричного шифрования. Система обеспечивает создание уникальной ключевой пары для каждого пользователя, где публичный ключ используется для шифрования данных, а приватный — для их дешифрования.

Интеграция с инфраструктурой обмена ключами: Публичные ключи пользователей регистрируются на сервере и становятся доступными для всех участников системы, что позволяет реализовать схему сквозного шифрования без необходимости предварительного обмена ключами по защищенным каналам. Этот подход существенно упрощает процесс установления защищенного соединения между пользователями.

Безопасная работа с криптографическими примитивами и кодированием: Все операции с ключами выполняются с использованием стандартных Java Security API, что гарантирует корректность реализации и устойчивость к известным атакам. Ключи кодируются в Base64 для удобства хранения и передачи, сохраняя при этом все криптографические свойства.

```
object RSAUtils {  
    private const val RSA_ALGORITHM = "RSA"  
  
    fun generateKeyPair(): Pair<String, String> {
```

```

val keyPairGenerator = KeyPairGenerator.getInstance(RSA_ALGORITHM)
keyPairGenerator.initialize(2048)
val keyPair = keyPairGenerator.generateKeyPair()

val publicKeyString = Base64.encodeToString(keyPair.public.encoded, Base64.NO_WRAP)
val privateKeyString = Base64.encodeToString(keyPair.private.encoded, Base64.NO_WRAP)
return Pair(publicKeyString, privateKeyString)
}

fun encryptText(text: String, publicKeyString: String): String? {
    return try {
        val publicKey = getPublicKeyFromString(publicKeyString)
        val cipher = Cipher.getInstance(RSA_ALGORITHM)
        cipher.init(Cipher.ENCRYPT_MODE, publicKey)
        val encryptedBytes = cipher.doFinal(text.toByteArray(Charsets.UTF_8))
        Base64.encodeToString(encryptedBytes, Base64.NO_WRAP)
    } catch (e: Exception) {
        null
    }
}

fun decryptText(encryptedText: String, privateKeyString: String): String? {
    return try {
        val privateKey = getPrivateKeyFromString(privateKeyString)
        val cipher = Cipher.getInstance(RSA_ALGORITHM)
        cipher.init(Cipher.DECRYPT_MODE, privateKey)
        val encryptedBytes = Base64.decode(encryptedText, Base64.NO_WRAP)
        val decryptedBytes = cipher.doFinal(encryptedBytes)
        String(decryptedBytes, Charsets.UTF_8)
    } catch (e: Exception) {
        null
    }
}

```

Гибкая обработка ошибок и nullable возвращаемые значения: Все методы шифрования и дешифрования возвращают nullable значения, что позволяет вызывающему коду корректно обрабатывать случаи сбоя криптографических операций. Этот подход особенно важен в мобильной среде, где доступные вычислительные ресурсы могут быть ограничены, а условия выполнения операций — нестабильными.

3.3.3 Интеграция криптографии в систему обмена сообщениями

Архитектура сквозного шифрования сообщений:

ChatRepositoryImpl реализует гибридную систему шифрования, где каждое сообщение защищается уникальным AES ключом, который в свою очередь шифруется RSA публичным ключом каждого получателя. Этот подход сочетает производительность симметричного шифрования с безопасностью асимметричной передачи ключей, обеспечивая высокий уровень защиты при минимальных накладных расходах.

Процесс отправки зашифрованного сообщения: Система выполняет последовательность операций, начиная с генерации сеансового AES ключа и вектора инициализации, шифрования текста сообщения, шифрования комбинированного ключа для каждого участника диалога, и завершая отправкой на сервер и локальным сохранением. Каждый этап включает проверки ошибок и обработку исключительных ситуаций.

Механизм дешифрования входящих сообщений: При получении сообщений система извлекает зашифрованный сеансовый ключ, дешифрует его с помощью приватного RSA ключа пользователя, разделяет полученную строку на компоненты AES ключа и вектора инициализации, а затем использует их для дешифрования содержимого сообщения. Этот процесс обеспечивает прозрачную для пользователя работу с зашифрованными данными.

```
class ChatRepositoryImpl @Inject constructor(  
    private val chatCloudDataSource: ChatCloudDataSource,  
    private val authRepository: AuthRepository  
) : ChatRepository {  
  
    override fun sendMessage(receiver: UserModel, message: String): Flow<UiState<MessageModel>> {  
        val aesKey = AesUtils.generateAESKey()  
        val aesIv = AesUtils.ivToString(AesUtils.generateRandomIV())  
        val encryptedMessage = AesUtils.encryptMessage(aesKey, message, aesIv)  
        val combinedKey = aesKey + aesIv
```

```

return chatCloudDataSource.sendMessage(
    receiver.userId,
    encryptedMessage,
    listOf(
        MessageKeyDto(
            receiver.userId,
            RSAUtils.encryptText(combinedKey, receiver.publicKey)
            ?: throw IllegalArgumentException()
        ),
        MessageKeyDto(
            authRepository.getAccessTokenModel().id,
            RSAUtils.encryptText(combinedKey, authRepository.getPublicKey())
            ?: throw IllegalArgumentException()
        )
    )
).map {
    when (it) {
        is ApiResponse.Success -> {
            val messageModel = decryptMessage(it.data)
            UiState.Success(messageModel)
        }
        is ApiResponse.Error -> UiState.Error(it.toString(context))
    }
}

```

```

override fun decryptMessage(messageDto: MessageDto): MessageModel? {
    val privateKey = authRepository.getPrivateKey()
    val combinedKey = RSAUtils.decryptText(messageDto.key, privateKey) ?: return null
    val aesIv = combinedKey.substring(combinedKey.length - 24, combinedKey.length)
    val aesKey = combinedKey.substring(0, combinedKey.length - 24)
    val decryptedMessage = AesUtils.decryptMessage(aesKey, messageDto.message, aesIv) ?: return
    null
}

```

```

return MessageModel(
    messageDto.id,
    messageDto.senderId,
    messageDto.receiverId,
    messageDto.senderId == myId,
    decryptedMessage,
    messageDto.createdAt
)

```

```
)  
}  
}
```

Обработка криптографических сбоев и преобразование данных:

Система включает комплексную обработку ошибок на всех этапах шифрования и дешифрования. В случае невозможности выполнить криптографическую операцию, вместо генерации исключения возвращается соответствующий результат ошибки, позволяя вышележащим слоям предпринять корректирующие действия. После успешного дешифрования данные преобразуются в модель представления с определением направления сообщения.

4. Выводы:

В ходе работы успешно разработан прототип безопасного мессенджера Private Messenger, демонстрирующий применение современных подходов Android-разработки. Приложение реализует полный цикл защищенной коммуникации с акцентом на безопасность данных и пользовательский опыт.

Архитектурные достижения: Создана многоуровневая система на базе паттерна MVVM с четким разделением ответственности. Внедрена реактивная архитектура через Kotlin Flow для управления состоянием, что обеспечило предсказуемость и тестируемость. Применение Hilt для dependency injection повысило модульность и упростило поддержку кода.

Ключевые реализованные функции: Разработана система аутентификации с JWT-токенами и автоматическим обновлением сессий. Создан модуль обмена сообщениями с использованием Paging 3 для эффективной работы с историей переписки. Реализовано сквозное шифрование на основе гибридной криптографии (AES-256 + RSA-2048), обеспечивающее конфиденциальность переписки. Интегрированы мультимедийные возможности через CameraX и ExoPlayer.

Освоенные технологии: В процессе работы применены современные Android Jetpack компоненты: ViewModel и LiveData для управления состоянием, Navigation Component для навигации, Room для работы с базой данных, Paging 3 для обработки больших наборов данных. Использованы Retrofit для сетевых запросов, корутины для асинхронного программирования, и криптографические API для обеспечения безопасности.

Технические инновации: Система реализует принцип perfect forward secrecy через уникальные сеансовые ключи для каждого сообщения.

Оптимизирована работа с памятью через эффективные адаптеры RecyclerView и стратегическое кэширование. Обеспечена корректная обработка различных конфигураций устройств через WindowInsets API.

Практическая значимость: Разработанное приложение представляет готовое решение для защищенной мобильной коммуникации, сочетающее промышленные стандарты безопасности с современным пользовательским интерфейсом. Проект демонстрирует комплексный подход к созданию полнофункционального Android-приложения и служит основой для дальнейшего развития.

Полученный опыт подтверждает возможность создания безопасных мессенджеров с использованием актуального стека технологий Android, соответствующих требованиям современной мобильной разработки.